

```

face_names = []
for face_encoding in face_encodings:

    matches = face_recognition.compare_faces(known_face_encodings, face_encoding, tolerance=0.5)
    name = "Unknown"
    x = sum(matches)
    if x == 1:
        first_match_index = matches.index(True)
        name = known_face_names[first_match_index]
    print name
    face_names.append(name)

```

Figure 6: Code for predicting the faces of the students

```

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

Figure 7: Code snippet to quit the real-time capture



Figure 8: Dataset (images) of class students

- These arrows are called gradients, which represent the flow from light to dark in the whole image.

The image contains a large number of pixels. Saving the gradient for every pixel is way too much detail to process. To get a higher-level basic flow of dark and light, this method breaks the image into small squares of 16x16 pixels each. In each square, the gradients that point in each major direction are counted (how many point upwards, up-right, right, etc). This square in the image is replaced with an arrow in the direction that is found to be strongest.

This encoding is done for all the images in the database (images used

for training). So, whenever a face is detected, it is compared with the known encoding, and the system then gives the name or roll number of the person. The given output, i.e., name or roll number, is the file name of the identified person in the known images.

Overall, the system is implemented in the Python language.

We used the images in the student database as the training data. In the snippet of code in Figure 4, lines 16 and 17 are the variable declarations of two lists for storing the face encoding of all the images of the students and their corresponding names in the database. From lines 21 to 25, each image is loaded



Figure 9: Face recognised as IMT2014053 (roll number)



Figure 10: Face recognised as IMT201401 (roll number)

and the corresponding encoding is stored in the list *known_face_encodings*.

After storing the face encoding of the training data, the code from lines 39 to 44 in Figure 5 triggers the camera for the real-time capture of the faces of the students. The algorithm processes each frame of the video, and starts storing the encoding of each face present within the frame (lines 47 to 50).

Later, it compares the encoding of each face in the frame with the encoding of the faces stored in the list *known_face_encodings*. If the encoding of the face in the frame matches with a known encoding, the system outputs the name of the file that matches the encoding (from lines 53 to 62 in Figure 6).

If it doesn't match with any encoding, then the system outputs the name as 'unknown'. Real-time capturing of images stops as soon as we press the button 'q', as shown in Figure 7.